# Bounded Initialization from a Tied Seed

## Arithmetic expressivity, whole-domain interpretations, and an audited small checker

Research prepared for Vladimir Reshetnikov

10 November 2026

**Abstract**

We give a tied-seed recursion theory with a numerical equality characteristic and literal-equation induction, and prove it whole-domain bi-interpretable with first-order intuitionistic primitive recursive arithmetic. The construction recovers bounded initialization, compiles each primitive-recursive description with internally proved cap coherence, and uses finite quantifier-free function graphs to preserve induction. Its standard term functions are exactly the primitive-recursive functions having a primitive-recursive unary majorant in one input coordinate. Separately, complete checker variants for the inherited calculus improve the observed byte, token, and AST minima of the supplied drafts, across different programs. Those implementation sizes are not attributed to the new arithmetic theory, and no absolute minimality or single program below all old coordinate minima is claimed.

**What is established, and what remains open.** The mathematical contribution is a route from a tied-seed recursor plus a numerical equality test to full first-order intuitionistic PRA, through bounded simulation and finite quantifier-free function graphs. The written proof establishes induction preservation and both whole-domain return interpretations. The implementation contribution is a complete checker of the inherited calculus at 612 normalized bytes (604 with the inherited compact list predicate), versus the current 616-byte minimum. These are different presentations. The small-checker figures are not assigned to the new arithmetic theory. The primary programs use 266 tokens, below the current 268-token minimum, but have 310/312 AST nodes. A separately qualified 639-byte ancillary variant improves the 300-node AST minimum to 298. The family envelope is therefore 604 bytes / 266 tokens / 298 AST nodes, achieved by different programs. An absolutely simplest axiomatization, an absolute size lower bound, and a single new checker below every old coordinate have not been established.

**Reading guide.** Section 1 lists every primitive logical and nonlogical ingredient of the new theory. The arithmetic development explains the bounded initialization construction and its interpretation argument. The expressivity section identifies the exact standard term-function class and the role of finite case distinctions. Section 7 independently states the measured calculus, implementation change, audit, and limits. The appendices preserve the inherited checker bridge with attribution. All arithmetic metatheorems are written proofs; no proof-assistant verification is claimed.

# Contents

**1 A fully explicit arithmetic basis** **3**

**2 Recovering bounded initialization internally** **5**

**3 A compiler with internal bounds and cap coherence** **7**

**4 Function graphs and preservation of induction** **9**

**5 Whole-domain bi-interpretability** **10**

**6 The exact term-function boundary** **11**

**7 A separately measured complete checker** **13**

**8 What has changed in the research problem** **18**

**9 Further research questions** **18**

**A Internal order, difference, and addition under a bound** **19**

**B Attribution of the inherited checker metatheory** **23**

**C Complementary introductions with one local assumption** **23**

**D Internal scalar predecessor bootstrap** **29**

**E Pairing, primitive recursion, and internal numerical composites** **33**

**F From literal-equation induction to full intuitionistic PRA** **34**

**G Arithmetic strength and both ordinary interpretation composites** **40**

# 1 A fully explicit arithmetic basis

This section specifies the proposed arithmetic theory independently of the measured checker in Section 7. Its distinguishing feature is the absence of an independently supplied initializer in its recursive constructor. The additional numerical operation is a Boolean test of actual equality.

## 1.1 Signature and finite staging

There is one numerical sort. The finite expression grammar is

$$t ::= x \mid 0 \mid St \mid \epsilon(t,t) \mid B(n;u,v.h).$$

In $B(n;u,v.h)$ only $h$ binds $u,v$; all its other variables are passive parameters. Expressions are finite and binders are renamed to avoid capture. For a strictly first-order reading, start with $0, S, \epsilon$ and introduce, at each stage, a function symbol $B_h(n,\vec{p})$ for each fixed earlier-stage term $h(u,v,\vec{p})$. The two equations below specify that symbol. Every proof uses finitely many such stages. There are no function variables, code variables, or a universally total evaluator.

Write $1 = S0$. The *tied seed* equations are

$$B(0;u,v.h) = S(h[0/u,0/v]), \tag{B0}$$
$$B(Sn;u,v.h) = h[n/u, B(n;u,v.h)/v]. \tag{BS}$$

Both substitutions on the right are simultaneous and capture-free. Thus the same fixed update body determines both the initial value and every subsequent transition.

The numerical test has exactly the following specifications:

$$\epsilon(x,y) = 0 \ \vee \ \epsilon(x,y) = 1, \tag{E-range}$$
$$\epsilon(x,y) = 1 \ \longleftrightarrow \ x = y. \tag{E-reflection}$$

The biconditional abbreviates its two implications, and is not an additional logical connective. Its equality on the right is the logic's actual equality, throughout the domain. These are universal axioms, not a test limited to standard numerals.

The remaining numerical axiom is

$$0 = 1 \ \longrightarrow \ \bot. \tag{Separation}$$

All free variables in the displayed axiom schemas are universally closed. Induction is restricted to one literal equation $E$:

$$\frac{\Gamma \vdash E(0,\vec{p}) \qquad \Gamma, E(z,\vec{p}) \vdash E(Sz,\vec{p})}{\Gamma \vdash E(t,\vec{p})}. \tag{EI}$$

The variable $z$ is absent from $\Gamma, t, \vec{p}$. Local assumptions in the second premise are discharged. In particular, the rule does not directly permit an implication, disjunction, or quantified induction matrix. Call this theory $T_{B\epsilon}$.

## 1.2 Successor can be eliminated from the primitive signature

The explicit successor above makes the computation laws easy to read, but it is not an essential primitive. Select the first-stage constant-body description

$$C(n,a) := B(n;u,v.a), \qquad \sigma(a) := C(0,a).$$

The constant-body instance of (B0) proves $\sigma(a) = Sa$. Thus one may start with only $0, \epsilon$, and the staged $B$ constructor, use $\sigma$ as the successor abbreviation, and write the computation schemas as

$$B_h(0, \vec{p}) = \sigma(h(0, 0, \vec{p})), \qquad B_h(\sigma(n), \vec{p}) = h(n, B_h(n, \vec{p}), \vec{p}).$$

Use $1 := \sigma(0)$ in the equality-test and separation axioms, and $E(\sigma z)$ in the induction rule. The constant-body symbol's own first equation is tautological; its successor equation and the other schemas remain substantive. No body is cyclic: $C$ has the earlier-stage body $a$, and the appearance of $C$ in an axiom is not an occurrence of its own definition inside that body.

Expanding $S$ by $\sigma$ gives an equivalent presentation on the same domain. Conversely, introducing $S := \sigma$ recovers the displayed theory. Recurrence uniqueness identifies the corresponding finite body descriptions. This removes one primitive constructor, but it is not by itself a complete-checker size estimate. The rest of the report retains the readable notation $S$.

### 1.3 The entire logical rule inventory

The ambient logic is ordinary intuitionistic first-order natural deduction with equality, and is counted as part of this presentation. Its formula constructors are

$$A ::= (s = t) \mid \bot \mid (A \wedge A) \mid (A \vee A) \mid (A \to A) \mid \forall x A \mid \exists x A.$$

Contexts are finite reusable lists. Hypothesis use is $\Gamma \vdash A$ for $A \in \Gamma$. Weakening, exchange, and contraction are allowed; equivalently they are admissible with reusable-context natural deduction. The logical rules are precisely the following, with retained contexts combined by union when needed:

| Constructor | Introduction and elimination |
|---|---|
| $\bot$ | From $\Gamma \vdash \bot$, infer $\Gamma \vdash C$. There is no introduction rule. |
| $\wedge$ | From $A$ and $B$, infer $A \wedge B$; from $A \wedge B$, infer either conjunct. |
| $\vee$ | From $A$, infer $A \vee B$; from $B$, infer $A \vee B$. From $A \vee B$, a derivation of $C$ under $A$, and a derivation of $C$ under $B$, infer $C$, discharging the two branch assumptions. |
| $\to$ | From $\Gamma, A \vdash B$, infer $\Gamma \vdash A \to B$, discharging $A$. From $A \to B$ and $A$, infer $B$. |
| $\forall$ | From $\Gamma \vdash A(z)$, infer $\Gamma \vdash \forall x A(x)$ when $z$ is absent from $\Gamma$ and the fixed parameters. From $\forall x A(x)$, infer $A(t)$. |
| $\exists$ | From $A(t)$, infer $\exists x A(x)$. From $\exists x A(x)$ and $\Gamma, A(z) \vdash C$, infer $\Gamma \vdash C$, discharging $A(z)$, with $z$ fresh for $\Gamma, C$ and the fixed parameters. |
| $=$ | Reflexivity $\Gamma \vdash t = t$. From $s = t$ and $F(s)$, infer $F(t)$ for every fixed formula template $F$, by capture-free substitution. |

Axiom instances may be used in every retained context. There are no classical logical axioms, excluded-middle schemas, additional induction rules, or unlisted reflection principles. The equality-test axioms imply decidability of numerical equality; they do not imply excluded middle for arbitrary quantified formulas.

Numerical coverage is derived below, after recovering quantifier-free induction; it is not an additional axiom. The falsity bridge remains explicit: positive equations alone admit a singleton ordinary structure, so a contradiction convention cannot silently replace the ordinary falsity axiom.

**The benchmark theory.** Here first-order iPRA has $0, S$ and symbols indexed by fixed finite primitive-recursive descriptions, their defining equations, successor injectivity and zero separation, and contextual induction for every quantifier-free formula in that language. Its logic is the same intuitionistic first-order equality logic just listed. The induction is not unrestricted induction on

arbitrary quantified formulas. Different descriptions may name the same numerical function; equality between such names is proved when needed, rather than decided by an assumed extensional program oracle.

## 2 Recovering bounded initialization internally

Write $1 = S0$. The only induction rule initially used is the contextual literal-equation rule (EI). The following construction does not assume that all standard-model identities about the new terms are axioms. Every assertion about the theory below is derived using its displayed computation equations, equality, intuitionistic logic, and the indicated instances of induction.

### 2.1 The inherited elementary macros

For a fixed finite body $h$, write $B_h(n, \vec{p})$ for $B(n; u, v.h(u, v, \vec{p}))$. The subscript is a fixed description, not an object-level function variable. All bound variables in a displayed macro are renamed apart before substitution. If substituting passive parameters creates a different finite body description, equation induction on the counter identifies its iterator with the specialized old iterator: the seeds agree and the step follows from the body identity and congruence. Thus the staged first-order presentation supplies substitution coherence; extensional equality of independently indexed descriptions is not assumed.

The tied-seed macros from the local-scopes draft give

$$\begin{aligned} D(n, a) &:= B(n; u, v.a), & W(n) &:= B(n; u, v.u), \\ P(n) &:= W(D(n, n)), & Z(n) &:= D(n, 0), \\ G(n, a) &:= P\big(P(B(a; u, v.D(Z(n), v)))\big). \end{aligned}$$

The two computation schemas prove

$$D(0, a) = Sa, \quad D(Sn, a) = a, \quad W(0) = 1, \quad W(Sn) = n,$$

and hence

$$P(0) = 0, \quad P(Sn) = n, \qquad Z(0) = 1, \quad Z(Sn) = 0. \tag{P,Z}$$

In particular successor is injective, by applying $P$ to an equality $Sx = Sy$. If $Sx = 0$, applying $P$ gives $x = 0$, and substitution then gives $S0 = 0$, contradicting separation. Thus every successor is nonzero.

Equation induction on $a$ gives

$$G(0, a) = 0, \qquad G(Sn, a) = a. \tag{Gate}$$

For the first equation the update in the inner $B$ is the identity, so its state remains one. For the second it is successor, so its state at counter $a$ is $S(Sa)$. These are separate literal-equation inductions with $n$ fixed as a passive parameter. The two outer predecessors then give the equations. No coverage principle is used here.

Define the fixed macro

$$L(n; u, v.h) := B\big(Sn; r, s.G(r, h[Pr/u, s/v])\big). \tag{L}$$

If the inner update is $H$, the gate equations give $H(0, s) = 0$ and $H(Sr, s) = h(r, s)$. Thus $B_H(0) = 1$, $B_H(1) = 0$, and the remaining steps are the desired updates. The theory proves

$$L_h(0, \vec{p}) = 0, \qquad L_h(Sn, \vec{p}) = h(n, L_h(n, \vec{p}), \vec{p}). \tag{L computation}$$

This is still zero-start recursion. An arbitrary initializer has not been smuggled into its definition.

### 2.2 Quantifier-free induction and coverage are derived

The equality-test axioms give, internally,

$$\epsilon(s,t)=1 \iff s=t, \qquad \epsilon(s,t)=0 \iff s\neq t. \tag{Reflection}$$

For the second equivalence, use the two-valued range axiom and separation $0\neq 1$. Thus every numerical equation is decidable, without assuming excluded middle for arbitrary formulas.

Every fixed quantifier-free formula $A$ has a term $\chi_A$ such that

$$\chi_A=0 \lor \chi_A=1, \qquad A \iff \chi_A=1. \tag{Characteristic}$$

This is an external recursion on finite formula syntax. Use $\epsilon(s,t)$ for an equality and zero for falsity. On Boolean inputs, $Z$ is negation and $G(b,c)$ is conjunction. Define disjunction by $Z(G(Zb,Zc))$ and implication by the Boolean expression for $\neg b\lor c$. Each formula is decidable by this construction, so the implication case is valid intuitionistically: for decidable $A$, $A\to B$ is equivalent to $\neg A\lor B$. Truth tables are checked by the two Boolean cases; the gate identities apply to 0 and 1.

Consequently contextual quantifier-free induction is admissible. Translate its base and step to the literal equation $\chi_A=1$, apply (EI), and translate back using (Characteristic). Arbitrary retained hypotheses remain unchanged. This derivation precedes all order and bounded-arithmetic arguments below.

Apply this derived rule to the quantifier-free formula

$$x=0 \lor x=S(Px).$$

The base is immediate; the successor case follows from $P(Sx)=x$. Thus the theory proves functional zero/successor coverage and hence

$$x=0 \lor \exists y\, x=Sy. \tag{Derived coverage}$$

Coverage is therefore not an additional axiom.

### 2.3 Order and bounded arithmetic

For a Boolean $b$, let $J(a,b):=B(Zb;u,v.a)$, so $J(a,0)=a$ and $J(a,1)=Sa$. Define

$$\begin{aligned} H(n,y) &:= L(n;r,v.J(v,\epsilon(r,y))),\\ d(x,y) &:= L(x;u,v.J(v,H(Su,y))),\\ \mathrm{mn}(x,y) &:= d(x,d(x,y)),\\ \mathrm{db}(n) &:= L(n;u,v.S(Sv)),\\ \mathrm{ad}_b(a,c) &:= d\big(d(\mathrm{db}(b),d(b,a)),d(b,c)\big). \end{aligned}$$

All five are terms in the original finite-description language. Use $x\le y$ as the quantifier-free abbreviation $d(x,y)=0$. The elementary order lemma proves that this is a discrete linear order, that $d$ is its truncated difference, and that mn is its minimum. It also proves, entirely by quantifier-free induction, the following bounded-addition identities:

$$a\le b \implies \mathrm{ad}_b(a,0)=a, \tag{1}$$

$$a,Sc\le b \implies \mathrm{ad}_b(a,Sc)=S\,\mathrm{ad}_b(a,c), \tag{2}$$

$$a,c\le b \implies \mathrm{ad}_b(a,c)\le \mathrm{db}(b), \tag{3}$$

$$\mathrm{ad}_b(b,b)=\mathrm{db}(b). \tag{4}$$

The function $c \mapsto \mathrm{ad}_b(a, c)$ is nondecreasing on $c \leq b$, when $a \leq b$. Only these bounded identities are used; there is no uncharged total binary addition term in the source language. The detailed inductions are given in Appendix A.

Define

$$\mathrm{sel}_b(c, a, e) := \mathrm{ad}_b\big(G(Zc, a), G(c, e)\big).$$

If $a, e \leq b$, coverage and the gate equations show that this returns $a$ at $c = 0$ and $e$ at every successor control. At most one summand is nonzero, so the result is again at most $b$.

For any fixed earlier update $h$, define the clipped initialized recursor

$$R_b(n, a; u, v.h) := L\big(Sn; r, s.\,\mathrm{sel}_b(r, \mathrm{mn}(a, b), \mathrm{mn}(h[Pr/u, s/v], b))\big). \qquad \text{(Clipped recursion)}$$

The gate, minimum, and $L$ equations give

$$R_b(0, a; h) = \mathrm{mn}(a, b), \tag{5}$$

$$R_b(Sn, a; h) = \mathrm{mn}\big(h(n, R_b(n, a; h)), b\big). \tag{6}$$

Quantifier-free induction gives $R_b(n, a; h) \leq b$ for every counter. The cap $b$ is a passive parameter throughout the recursion. This supplies arbitrary initialization whenever an independent cap has first been fixed. It is compatible with the one-coordinate growth obstruction: the output is bounded in the cap coordinate.

## 3 A compiler with internal bounds and cap coherence

Fix the usual finite primitive-recursive descriptions built from zero, successor, projections, composition, and primitive recursion. The following theorem is an external construction on descriptions, producing finite source terms and finite source proofs. It does not add a uniform evaluation function for arbitrary description codes.

**Theorem 3.1** (Internal bounded realization)**.** *For every fixed $k$-ary primitive-recursive description $f$, one can construct source terms*

$$\tau_f(b, x_1, \ldots, x_k), \qquad M_f(b)$$

*and proofs in $T_{B\epsilon}$ with the following properties.*

1. *$M_f$ is nondecreasing and $b \leq M_f(b)$.*
2. *The bound $\tau_f(b, \vec{x}) \leq M_f(b)$ holds for all inputs, whether or not the inputs are bounded by $b$.*
3. *If every $x_i \leq b$ and every $x_i \leq c$, then $\tau_f(b, \vec{x}) = \tau_f(c, \vec{x})$.*
4. *The finite-case graphs built from these terms satisfy the defining equations of the description $f$, as proved below.*

*All comparisons are the quantifier-free source abbreviations just defined. The construction produces proofs on the whole domain, not just correctness on standard numerals.*

*Proof.* We construct the first three items by induction on the fixed description. The fourth will then follow from the actual defining formulas.

**Initial descriptions.** For zero use $\tau_0 = 0$; for a projection use $\tau_{\pi_i}(b, \vec{x}) = \mathrm{mn}(x_i, b)$; for successor use $\tau_S(b, x) = S\,\mathrm{mn}(x, b)$. The common bound $M(b) = Sb$ works. The minimum identities prove bounds and coherence immediately.

**Composition.** Suppose $f(\vec{x}) = h(g_1(\vec{x}), \ldots, g_m(\vec{x}))$. Write $G_i = M_{g_i}$ and $H = M_h$. Put

$$Q = G_1 \circ \cdots \circ G_m, \qquad M_f = H \circ Q,$$

with $Q$ the identity if $m = 0$. Since all $G_i$ are nondecreasing and inflationary, $G_i(b) \le Q(b)$ for each $i$. Define

$$\tau_f(b, \vec{x}) = \tau_h\big(Q(b), \tau_{g_1}(b, \vec{x}), \ldots, \tau_{g_m}(b, \vec{x})\big).$$

The global bound follows from the bound for $\tau_h$. For coherence, the inner values agree at any two input caps by their induction hypotheses. They are bounded by both $Q(b)$ and $Q(c)$; applying coherence for $h$ proves the result. Monotonicity and inflationarity of $M_f$ follow by composition.

**Primitive recursion and its majorant.** Suppose

$$f(0, \vec{x}) = g(\vec{x}), \qquad f(Sn, \vec{x}) = h(n, f(n, \vec{x}), \vec{x}).$$

Let $G = M_g$, $H = M_h$, and put

$$F = H \circ G \circ S.$$

This is a source unary term with internal proofs of monotonicity and $Sz \le F(z)$. Let

$$I_F(t) := L(t; u, v.F(v)), \qquad C_b := I_F(S\,\mathrm{db}(b)), \qquad M_f(b) := C_b. \tag{Majorant}$$

Equation induction gives $I_F(0) = 0$, $I_F(St) = F(I_F(t))$, and $t \le I_F(t)$. Its monotonicity is a quantifier-free induction using order discreteness: if $s \le St$, either $s = St$ or $s \le t$. Consequently $b \mapsto C_b$ is nondecreasing and inflationary. This construction uses only successor, composition, and zero-start iteration for its majorants.

For $j \le b$, use the following actual source term for the stage bound:

$$K_b(j) := I_F\big(S\,\mathrm{ad}_b(b, j)\big). \tag{Stage bound}$$

The bounded-addition identities prove

$$b, G(b) \le K_b(0), \tag{7}$$

$$Sj \le b \implies K_b(Sj) = F(K_b(j)), \tag{8}$$

$$j \le b \implies b \le K_b(j) \le C_b. \tag{9}$$

For example, $K_b(0) = I_F(Sb) = F(I_F(b)) \ge G(b)$ because $I_F(b) \ge b$, $G$ is nondecreasing, and $H$ is inflationary. The upper bound uses $\mathrm{ad}_b(b, j) \le \mathrm{db}(b)$. Also $H(z) \le F(z)$, by inflationarity of $G$ and monotonicity of $H$.

Write $n_b = \mathrm{mn}(n, b)$ and $\vec{x}_b = (\mathrm{mn}(x_i, b))_i$. Define

$$\tau_f(b, n, \vec{x}) := R_{C_b}\big(n_b, \tau_g(b, \vec{x}_b); u, v.\tau_h(C_b, u, v, \vec{x}_b)\big). \tag{Compiled recursion}$$

Its global bound is immediate from the clipped-recursion bound.

**No clipping on bounded inputs.** Fix passive parameters $\vec{x} \le b$ and let

$$r_b(j) := R_{C_b}\big(j, \tau_g(b, \vec{x}); u, v.\tau_h(C_b, u, v, \vec{x})\big).$$

We prove by quantifier-free induction on $j$ that

$$j \le b \longrightarrow r_b(j) \le K_b(j). \tag{Invariant}$$

At zero, $\tau_g(b,\vec{x}) \le G(b) \le K_b(0) \le C_b$, so the seed is not clipped. For the step assume $Sj \le b$. Then the induction hypothesis applies at $j$; all arguments $j, r_b(j), \vec{x}$ of the step function are bounded by $K_b(j)$, as well as by $C_b$. Coherence for the already constructed description $h$ gives

$$\tau_h(C_b, j, r_b(j), \vec{x}) = \tau_h(K_b(j), j, r_b(j), \vec{x}) \le H(K_b(j)) \le F(K_b(j)) = K_b(Sj) \le C_b.$$

Thus its value is not clipped and the invariant follows. The same argument establishes the actual untruncated step identity

$$Sj \le b \implies r_b(Sj) = \tau_h(C_b, j, r_b(j), \vec{x}). \qquad \text{(Unclipped step)}$$

The matrix of the induction is quantifier free, and the input cap and all passive parameters remain fixed throughout it.

**Coherence between two arbitrary caps.** Fix $b, c$ and inputs $n, \vec{x}$ bounded by both. After the minimum identities remove their input clipping, compare the two runs $r_b, r_c$. Quantifier-free induction on $j$ proves

$$(j \le b \wedge j \le c) \longrightarrow r_b(j) = r_c(j). \qquad \text{(Two-cap invariant)}$$

The base uses coherence for $g$ and the already proved absence of seed clipping. If $Sj \le b, c$, the two states at $j$ are equal by the induction hypothesis and bounded by both $C_b, C_c$. The other step arguments are bounded by both caps too. Coherence for $h$ equates the two raw step values. Formula (Unclipped step) removes their clipping, giving the successor equality. At $j = n$ this is exactly the required coherence of $\tau_f$. No induction on a universally quantified formula has occurred: $b, c, n, \vec{x}$ are fixed parameters, and both displayed invariant matrices are quantifier free.

Every recursive step body in this construction is the already compiled original-language term $\tau_h$, inlined before it is inserted into $L$ or $R_b$. No recursion schema is extended for free to an opaque function adjoined later by a graph definition. This distinction prevents a conservative-definition argument from hiding a stronger recursion principle.

This completes the construction of the terms and internal proofs. □

## 4 Function graphs and preservation of induction

For a nonempty tuple $\vec{x} = (x_1, \dots, x_k)$ let

$$C_i(\vec{x}) := \bigwedge_{j<i} x_j < x_i \ \wedge \ \bigwedge_{j>i} x_j \le x_i.$$

This says that coordinate $i$ is the first largest one. Discrete linearity proves that these fixed finitely many quantifier-free cases are exhaustive and disjoint. Define

$$\Phi_f(\vec{x}, z) := \bigvee_{i=1}^{k} (C_i(\vec{x}) \wedge z = \tau_f(x_i, \vec{x})). \qquad \text{(Graph)}$$

For a nullary description use $z = \tau_f(0)$.

**A concrete example: binary addition.** The bounded-addition term already illustrates the method without the full compiler. Its total single-valued graph is

$$\Phi_+(x, y, z) := (y \le x \wedge z = \mathrm{ad}_x(x, y)) \ \vee \ (x < y \wedge z = \mathrm{ad}_y(x, y)).$$

On either case the selected cap bounds both arguments. The bounded-addition identities and coherence under changing a valid cap give the usual addition equations. Each possible output term is bounded by twice one input, while addition itself is not bounded by any unary function of one input. Thus graph interpretation succeeds exactly where a single-term definition is obstructed.

**Lemma 4.1** (Totality, uniqueness, and computation)**.** *The theory proves that every $\Phi_f$ is total and single-valued. In the definitional extension by its uniquely specified function $f^*$, the defining composition and primitive-recursion equations of the description $f$ hold. The extension admits quantifier-free induction for formulas containing finitely many of these defined functions.*

*Proof.* Exhaustiveness provides a displayed value term on each case, proving existence. Disjointness proves uniqueness. Moreover, for every source cap $b$ bounding the arguments, coherence proves

$$f^*(\vec{x}) = \tau_f(b, \vec{x}). \qquad \text{(Cap realization)}$$

One proves this by the finite graph cases: on case $i$, both $b$ and $x_i$ bound all arguments, so the corresponding values are equal.

Initial equations follow from the compiled initial descriptions. For composition, use any cap $b$ bounding the common input tuple. The compiled inner values are bounded by $Q(b)$, so applying (Cap realization) at $Q(b)$ to the outer function proves the composition equation directly from its compiler definition.

For recursion at counter zero, choose a graph case whose cap bounds $0, \vec{x}$. The seed bound prevents clipping, so the value is $\tau_g(b, \vec{x}) = g^*(\vec{x})$. At a successor counter, choose a case whose cap $b$ bounds $Sn, \vec{x}$. Then $n \leq b$, the unclipped-step identity applies, and (Cap realization) identifies the previous state with $f^*(n, \vec{x})$. All step arguments are bounded by $C_b$; the same cap realization for $h$ identifies the update with $h^*(n, f^*(n, \vec{x}), \vec{x})$. This proves the recursion equations.

It remains to justify the induction claim, rather than merely invoke conservativity of definitional extensions. Every occurrence of a defined function has a graph given by finitely many quantifier-free cases and explicit source value terms. By external induction on a finite term, it therefore has a finite family of pairs $(A_j, t_j)$, with source quantifier-free guards $A_j$ and source terms $t_j$, such that the guards cover all inputs and the term equals $t_j$ on guard $A_j$. At a composition of terms take the finite Cartesian product of the argument cases and then apply the finitely many cases of the outer function. This introduces no object-language existential quantifier. Guards need not be disjoint after expansion; their associated values agree on every overlap.

For a quantifier-free formula involving defined functions, take a common finite refinement of the cases of all its terms. On each guard replace its terms by their source values and retain its actual intuitionistic connectives. The finite disjunction of these guarded source formulas is quantifier free and is provably equivalent to the original formula. Covering cases justify the equivalence also for implication and negation. Applying source quantifier-free induction to this equivalent formula transfers induction to the expanded formula. This is the essential finite-case step: an arbitrary existential graph translation would not suffice. □

## 5 Whole-domain bi-interpretability

**Theorem 5.1** (Arithmetic equivalence)**.** *The explicitly axiomatized theory $T_{B\epsilon}$ and full first-order intuitionistic primitive recursive arithmetic are bi-interpretable. Both interpretations use the whole numerical domain and actual equality. Their composite definable isomorphisms are identity maps. In particular the theories are equiconsistent, and neither an appeal to standard-model agreement nor induction on quantified matrices is needed for this conclusion.*

*Proof.* **Interpretation in iPRA.** Interpret $0, S$ ordinarily and $\epsilon$ as iPRA's primitive-recursive equality characteristic. Translate every fixed staged $B_h$ symbol by primitive recursion with initial value $Sh(0, 0, \vec{p})$ and update $h(n, v, \vec{p})$, first translating its earlier-stage body. The two computation equations become defining primitive-recursion equations. The equality-test, separation, and contextual equation-induction principles hold in iPRA. Logical connectives, quantifiers, and genuine falsity retain their ordinary intuitionistic meanings.

**Interpretation of iPRA in the source.** Interpret each fixed primitive-recursive function description by its total unique graph $\Phi_f$ and use ordinary relational interpretation of function terms, or equivalently work in the conservative definitional extension by the functions $f^*$. The graph computation lemma proves every defining equation. The source predecessor and separation prove the successor boundary laws. The finite-case quantifier-free elimination in that lemma proves every translated iPRA induction instance. Thus all axioms and inference rules of full iPRA are interpreted, with their intuitionistic logic unchanged.

**The composite on the iPRA side.** An externally fixed primitive-recursive description is uniquely determined by its initial and step equations. External induction on its finite description, with ordinary quantifier-free induction on the recursion counter at each recursion node, proves in iPRA that its composite interpretation equals the original function. The necessary graph functions can be treated in their conservative definitional extensions; finite-case elimination supplies any needed induction instances. Hence the identity on the domain preserves every primitive-recursive function and actual equality.

**The composite on the source side.** Proceed by external induction on the finite stage of a source description. The ordinary $0, S$ operations are preserved. The interpreted equality characteristic and the original $\epsilon$ are both Boolean characteristics of actual equality; their range and reflection properties prove that they agree by the two Boolean cases. For a fixed $B_h$, the earlier-stage induction hypothesis identifies the interpreted update with its original source body $h$. Both the original function and its composite satisfy (B0) and (BS) with that same body. Induction on the equation asserting their equality proves recurrence uniqueness and identifies them at every counter and every passive parameter. If the composite is expressed by defined graph functions, the finite-case elimination lemma again legitimizes this induction in the original theory. Thus the identity map preserves every source function symbol too.

Since all numerical operations and equality are preserved, external induction on arbitrary formula syntax proves preservation of all logical formulas under both composites. Quantifiers range over the same whole domain. This establishes the two identity definable isomorphisms and the claimed full bi-interpretation. Genuine falsity is preserved in both directions, giving equiconsistency. □

**Status and size.** This is a written mathematical construction with explicit induction invariants and a finite-description compiler. It is not a Lean or Rocq formalization. It establishes a different arithmetic basis; it does not establish that a complete proof checker for this basis is smaller than the current compact Choice-DI implementations. The equality-test axioms, ordinary logical rules, and all validation code must be charged in any implementation comparison.

## 6 The exact term-function boundary

The preceding theorem interprets all PR functions by finite graphs. It does not say that every multivariate PR function becomes a term. The distinction can be made exact, with no infinite family

of primitive unary operations and no oracle for arbitrary bounded predicates.

For $k \geq 1$ let $\mathcal{A}_k$ consist of the primitive-recursive functions $f : \mathbb{N}^k \to \mathbb{N}$ for which there are a coordinate $i$ and a unary primitive-recursive function $F$ satisfying

$$f(\vec{x}) \leq F(x_i) \quad \text{for all } \vec{x} \in \mathbb{N}^k.$$

Let $\mathcal{A}_0$ be the nullary constants. The primitive-recursive qualification on the majorant is essential. The established term *almost unary* usually allows an arbitrary unary bound; see [5]. The class here keeps the stronger effective bound.

**Theorem 6.1** (Exact standard term functions)**.** *The standard numerical functions represented by the finite $0, S, \epsilon, B$ language are exactly $\mathcal{A}_k$ at each arity $k$.*

*Proof.* For the forward inclusion use the one-coordinate majorant induction underlying the previous tied-seed obstructions [4, 2]. We restate its actual arithmetic cases. The equality characteristic is bounded by one, and successor is unary. Replace every unary PR bound by its nondecreasing PR envelope when necessary. For a body $h(u, v, \vec{p})$, a bound $H(p_j)$ in one passive coordinate bounds the output of $B_h$ by $1 + H(p_j)$. A bound $H(u)$ in the counter coordinate bounds it by $1 + H(n)$ at counter $n$. A bound $H(v)$ in the state coordinate bounds it by

$$K^n(1 + H(0)), \qquad K(z) = \max(z, H(z)),$$

a unary PR function of $n$. If the actual counter is a term, compose with its own one-coordinate bound. Ordinary term substitution preserves the invariant by taking a nondecreasing envelope of the outer bound and following its controlling argument. Every fixed term is PR because its descriptions are finite. This proves the upper inclusion.

For the converse let $f(\vec{x}) \leq F(x_i)$. The internal compiler already supplies quantifier-free partition cases $C_j(\vec{x})$ and source value terms

$$v_j(\vec{x}) = \tau_f(x_j, \vec{x}),$$

whose value is $f(\vec{x})$ on case $C_j$. Their characteristic terms $\chi_{C_j}$ are supplied by the derived Boolean compilation. Put

$$a_j(\vec{x}) = G(\chi_{C_j}(\vec{x}), v_j(\vec{x})).$$

This is zero off its case and equals $f$ on its case. Exactly one case holds, so every $a_j$ is bounded by $F(x_i)$, and at most one is nonzero.

Every unary PR function is already a source term: in the one-variable instance of the compiler, the sole maximum case is automatic and the value is $\tau_F(x, x)$. Use this term to form $b = F(x_i)$. Starting with $s_0 = 0$, form the fixed finite sequence of terms

$$s_j = \mathrm{ad}_b(s_{j-1}, a_j) \quad (1 \leq j \leq k).$$

Every partial sum is either zero or $f$, so both arguments of each bounded addition are at most $b$. The last value is $f$. At arity zero use its numeral. The construction takes a fixed description of $f$, a coordinate $i$, and a valid unary PR majorant as input; it does not decide whether an arbitrary proposed bound is valid. □

**A sharp finite-choice bound.** For every $k$-ary PR function, the internal compiler uses at most $k$ possible output terms, one for each first-maximum case. This is optimal in general for this finite-term selection interface. Suppose the $k$-variable sum were always one of $r < k$ source terms. Each term

is bounded by a unary function of one controlling input. Choose an input coordinate that controls none of the $r$ terms, fix all other coordinates at zero, and increase the chosen one. All candidate terms remain bounded by fixed constants, while the sum is unbounded. This is a contradiction.

The claim is about selecting from a finite family of output terms. It is not a lower bound on the number of clauses in an arbitrary graph formula: a Boolean characteristic of any PR graph is itself globally bounded and hence, by Theorem 6.1, a source term. Such a graph can therefore have a one-equation definition. The compiler's finite output witnesses are what provide totality and preserve induction internally.

### 6.1 Why the effective majorant qualification matters

The class just characterized is strictly smaller than the intersection of PR functions with the ordinary set-theoretic almost-unary class.

Enumerate the unary PR descriptions as $u_0, u_1, \ldots$ and define the total recursive function

$$g(x) = 1 + \max_{e \le x} u_e(x).$$

Its computation terminates because every listed description is total. It has no PR majorant: any proposed unary PR bound occurs as some $u_e$, and $g(x) > u_e(x)$ for every $x \ge e$. A fixed machine computes $g$. Let $s(x,t)$ return its output if it halts within $t$ steps on input $x$, and return zero otherwise. Bounded machine simulation is primitive recursive, so

$$f(x,t) = x + s(x,t)$$

is PR. For fixed $x$ its range is contained in $\{x, x + g(x)\}$, so it is almost unary with an unrestricted bound in $x$. A unary PR bound in $x$ would majorize $g$, which is impossible. No bound in $t$ is possible at all, because $f(x,t) \ge x$. Thus $f$ belongs to the ordinary almost-unary PR intersection but not to $\mathcal{A}_2$.

This example distinguishes standard function classes. The full iPRA bi-interpretation rests on the preceding internal compiler and its finite-case induction argument, rather than on standard-function expressivity alone.

Likewise, the converse in Theorem 6.1 assumes that the proposed majorant is valid in the standard model. To prove internally that its resulting single term equals the graph-defined function, one additionally needs an internal proof of that majorant. No such extra hypothesis is required for the finite-case bi-interpretation.

## 7 A separately measured complete checker

The deliverable also improves the observed implementation coordinates across a family of programs. It combines two already-present ideas: the arity representation from [3] and the direct predecessor packets from [2]. This is a synthesis of their certificate interfaces. It is not a size measurement for $T_{B\epsilon}$, whose arithmetic and logical basis differs.

### 7.1 A pinned and inclusive comparison

The repository snapshot is `fb2dbd2a33c6b132671a54b029d0bf3cd98c633d`. Its directory contains thirteen direct ZIP archives. Each input was verified against its Git blob identity. The recursive

inventory contains 370 distinct Python source byte strings, including 165 nominal standalone public drivers. Every non-driver below one of the observed coordinate minima was inspected; these are supporting scripts, not overlooked complete checkers. The full source census and input identities are in `evidence/`.

The old coordinate minima are 616 normalized bytes, 268 significant tokens, and 300 AST nodes, achieved by different programs. They are an observed finite-corpus envelope, not a mathematical lower bound over all possible proof checkers. Ancillary programs and matched comparison files are included. In particular, the earlier 672/277/311 figures quoted in several draft abstracts are no longer the correct baseline for this snapshot.

| Complete program | Normalized bytes | Tokens | AST | Raw bytes | Gzip-9 |
|---|---|---|---|---|---|
| Current byte leader | 616 | 269 | 311 | 616 | 366 |
| Current AST/token leader | 649 | 268 | 300 | 648 | 367 |
| Delivered explicit-list checker | 612 | 266 | 310 | 612 | 360 |
| Delivered compact predicate variant | 604 | 266 | 312 | 604 | 356 |
| Ancillary AST profile | 639 | 267 | 298 | 639 | 360 |

The 612-byte implementation uses ordinary `isinstance(a,list)`. The 604-byte variant replaces this with the inherited exact-JSON-list predicate `a*0==[]`; it has a slightly larger AST. Both recursively traverse finite arrays and do not depend on malformed nonempty strings recursively iterating into themselves. The table counts the whole executable module: input, traversal, rules, scope handling, and success verdict. Python, the standard JSON implementation, and the runtime are the common infrastructure of the comparison.

Normalization removes genuine module/function/class docstrings, uses `ast.unparse` on the entire module, and appends a newline. Comments disappear under this operation. Tokens exclude encoding, indentation, line separators, and end markers; AST counts include the complete module. Gzip uses level 9 with timestamp zero and no original filename. Its role is informational, not an optimization objective. Exact sources, compressed sources, Python/zlib versions, hashes, and a reproducer are supplied.

The primary and compact implementations improve bytes and tokens. The ancillary AST profile improves the AST minimum under the same finite-CPython-process qualification as the old AST leader. The delivered family therefore has coordinate minima 604/266/298, compared with 616/268/300 for the pinned corpus. Both triples combine different programs. No single delivered program is below all three old minima, and the small implementation deltas do not by themselves establish a significantly simpler arithmetic foundation.

**Qualification of the ancillary profile.** `code/checker_ast.py` uses the inherited exception-dispatch opener. On canonical lists and numerical leaves, successful opening is exactly the primary opening. A child exception cannot make a parent list succeed as a numerical leaf, since its fallback comparison raises a type error. Empty strings and empty dictionaries are zero/truth aliases; a dictionary whose sole key is empty is a successor-zero/paired-truth alias, with its value ignored. Nonempty strings and nonempty dictionary keys reject through CPython's finite recursion limit. This is therefore not a structurally terminating parser over all finite JSON in an ideal interpreter with an unbounded stack. The recommended explicit-list implementation does not depend on this behavior.

The AST profile also initializes the global freshness floor in the opener's default-depth expression and installs the input-pop method in the initial call. These expression assignments execute before either value is used; the stored default depth remains zero. Its inference code is otherwise identical. The supplied profile review includes 146 passing canonical, alias, malformed-input, and runtime-contract

controls. These qualifications are part of its comparison with the already exception-based current AST leader.

### 7.2 The exact mathematical calculus being checked

The inherited numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; v.h),$$

with a separately supplied initializer $a$ and update binding only its scalar state. Its computation schemas are

$$I(0, a; v.h) = a, \qquad I(Sn, a; v.h) = h[I(n, a; v.h)/v].$$

Native formulas are equations, $Q_y A$, and $D_{x,y} A$, interpreted as

$$Q_y A := A(0) \to \forall y A(y), \qquad D_{x,y} A := \exists x \big(A(x, 0) \to \forall y A(x, y)\big).$$

The complete rule inventory is hypothesis use; the two computation schemas; equality transport; and

$$\frac{\Gamma \vdash Q_y A \quad \Gamma \vdash A(0)}{\Gamma \vdash A(t)}, \qquad \frac{\Gamma \vdash D_{x,y} A \quad \Gamma, Q_y A(b, y) \vdash F}{\Gamma \vdash F},$$

$$\frac{\Gamma, A(t, 0) \vdash A(t, Sz)}{\Gamma \vdash D_{x,y} A}, \qquad \frac{\Gamma, E(t, z) \vdash E(t, Sz)}{\Gamma \vdash D_{x,y} E}.$$

The last matrix $E$ is a literal equation. The usual retained-context freshness conditions apply to $b, z$, and local hypotheses are discharged. Equality transport allows any fixed native formula template. No ordinary logical connective, constructor separation, reflexivity, or extra induction rule is silently added to this native calculus.

The reproduced appendices derive ordinary intuitionistic logic by numerical selectors, derive actual equality reflection and quantifier-free induction, and compile every fixed PR description. They prove both whole-domain interpretation composites using the identity relation $x = y$. This is the inherited metatheory of this checker, explicitly attributed; the new contribution here is its combined packet encoding.

### 7.3 The raw interface and its conservative meaning

The input is one JSON array. Its last element is the requested closed formula; preceding four-field instructions are consumed from right to left. An unused prefix is inert. Hypothesis closure consumes no instruction. Acceptance means both exit status zero and the sole line `ACCEPT`; parsing or checking exceptions reject by nonzero exit. This is the same driver convention used by the actual current minimum programs.

At formula positions, arrays of arity zero, one, two, and three denote respectively truth, $D$, equality, and $Q$. Arity at least four denotes truth. Canonical $D$ is $[A]$, and canonical $Q$ is $[A, A, A]$, with its first two entries redundant annotations. At numerical positions, an empty array denotes zero, unary arrays denote successor, binary arrays denote zero annotations, and ternary arrays denote initialized iteration. Larger arrays denote zero. These aliases are interpretations of finite raw syntax, not additional mathematical symbols.

A numerical root has opening depth three; a formula root has depth zero. At every array the traversal adds $5 - \text{arity}$. Thus the meaningful increments at arities one, two, and three are $4, 3, 2$. They

respectively supply the paired binder, the transition from formula to numerical equality, and the closure/state slot. Dummy numerical slots have value zero. Negative coordinates identify already admitted rigid parameters; positive coordinates identify protected slots or the distinguished exposed hole. The canonical serializer and the inherited opening proof give the precise slot convention. Successfully traversed irrelevant annotations have no effect on the interpretation.

The global lower bound on fresh parameter names decreases at each continuation. A major proof receives the unchanged context; extending its own continuation never mutates the caller's context. An inserted numerical image is itself traversed at numerical root. The semantics and conditional opening invariant, including noncanonical aliases, are reproduced in `reference/arity_checking_argument.tex`. A blanket claim that every raw alias is idempotently validated would be false below ignored high-arity nodes; that claim is not needed.

### 7.4 The combined arithmetic instruction

A new instruction has the form

$$[\texttt{null}, p, a, h].$$

It generates the local equation

$$\begin{cases} I(0, a; v.h) = a, & p \text{ has a false raw representation,} \\ I(Sp, a; v.h) = h[I(p, a; v.h)/v], & p \text{ has a true raw representation.} \end{cases}$$

The successor on the left is constructed by the checker. The branch is a syntactic representation test, not a numerical positivity assertion in a model.

**Lemma 7.1** (Conservative composition of the supplied interfaces)**.** *The new arithmetic packet* $[\texttt{null}, s, u, t]$ *constructs the same pre-opening equation as the old arity-calculus packet* $[0, [s, u, t], s, 0]$*. The new continuation opens it at zero. It is therefore an instance of the old arithmetic supply's proved zero-instance case.*

*Proof.* Both source constructors produce

$$[[\, s \text{ and } [s], u, t\,],\ i(t, [s, u, t], 3) \text{ if } s \text{ else } u].$$

The only relevant continuation difference is the null major: it is false and is not numerically equal to an arity parity, so the new continuation's image is zero. The arity-calculus supply lemma proves precisely the zero-completed computation equation, including validation of the recursive image when the update uses its state. Consequently adjoining it is sound. This is a correspondence of concrete source constructors, not an assumption that every newly exposed arithmetic parameter is arbitrary. □

Literal-zero successor predecessors are recovered by the familiar nonempty box $J(p) = I(0, p; v.0)$ with $J(p) = p$, followed by transport. Hence every mathematical successor computation has a canonical certificate. The new split on a null major also admits literal-zero introduction witnesses directly.

All other continuations retain the arity-calculus justification. A proved unary $D$ major multiplied by three exports its $Q$ closure; other nonempty array majors export only high-arity truth. The introduction call `i(*a,t,2)` admits a one-child $D$ goal or an already-true empty array; an array with more children supplies too many positional arguments and rejects. Only a literal equation matrix receives progress induction. Equality transport checks both the major pattern and the opened target. Closure specialization retains the zero-source guard. These conditions are essential to the inherited soundness proof.

**Theorem 7.2** (Scope of the delivered checkers)**.** *On canonical finite certificates, the delivered programs check exactly the inherited native consequence relation, subject to ordinary finite implementation resource limits and the stated CPython qualification for the ancillary AST profile. Every successful raw certificate has the inherited conservative decoding, extended by the ancillary iterable aliases where applicable. Therefore the mathematical calculus checked is equiconsistent and whole-domain bi-interpretable with first-order* iPRA*.*

*Proof.* The lemma handles the only changed arithmetic source. The unchanged opening, freshness, and logical continuation arguments handle the remaining branches. The ancillary profile's successful canonical traversal agrees with the primary traversal, and its additional iterable aliases have the meanings just specified. Canonical arithmetic completeness follows from boxing zero; the remaining canonical proof compilation is inherited. Finally apply the reproduced native/ordinary arithmetic bridge. This is a written metatheorem, not a claim of Lean or Rocq verification. □

### 7.5 Executable evidence and limits

The supplied fixtures cover state-dependent computation, zero predecessors, genuine induction, nonvacuous zero witnesses, nested transport, compound case introduction, neutral malformed endpoints, and rejection of compound induction. All three compact profiles and the readable implementation are replayed under ordinary Python and Python `-O`; their checks do not use removable assertions. The main checker suite has 172 passing expectations. Separate tests execute the actual tied-seed numerical macros and compare their finite values to independent ordinary-arithmetic oracles.

The reproducible numerical audit has 1,102 passing finite expectations and no skipped cases. It validates the displayed bounded macros over the stated small ranges; it does not establish their universal induction proofs or model-theoretic interpretation composites. The package makes no claim that a few examples constitute proof-assistant verification, and supplies no automatic translator for arbitrary ordinary-iPRA proofs.

### 7.6 Entire recommended trusted program

```
import json
k = 0
i = lambda a, t=None, d=0: [i(b, t, d + 5 - len(a)) for b in a] if isinstance(a, list) else a if k <=
    a < d else d % 2 and i(t, d=3)

def c(a, g):
    global k
    if i(a) in g:
        return
    m, s, u, t = q()
    if m:
        c(m, g)
        if m in ([s, u], not s and [t] * 3) and i(t, u) == a:
            a = i(t, s)
        h = m * 3
    elif m is None:
        h = [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]
    else:
        h = a = i(*a, t, 2)
    k -= 1
    c(i(a, [k]), g + [i(h, k if m or len(a) % 2 == m else 0)])
q = json.load(open(0)).pop
c(q(), [])
print('ACCEPT')
```

## 8 What has changed in the research problem

An almost-unary term bound remains a valid obstruction to a direct term definition of addition or unrestricted initialized recursion. It does not, by itself, obstruct an interpretation by finite function graphs. Adding a numerical equality characteristic changes the available finite computations enough to construct bounded subtraction, selection, and recursive simulation, while the term algebra can still obey the original one-coordinate growth restriction. This is the reason a graph interpretation can succeed without contradicting the term obstruction.

The result depends on a genuine additional numerical primitive with whole-domain range and reflection axioms. Logical equality in formulas is not an interchangeable substitute for a term returning its Boolean value inside a recursive update. Likewise, a statement that all unary PR functions are available would hide the work of generating and verifying them. The bounded compiler avoids that assumption.

There are still two separate optimization costs. The new arithmetic basis trades an independent initializer for an equality characteristic and its specifications; it is not automatically cheaper to validate. The measured small checker keeps the old initialized iterator and gains only from composing previously established encodings. Both improvements are useful, but neither justifies assigning the other's cost or novelty to it.

## 9 Further research questions

1. **A genuinely smaller checker for $T_{B\epsilon}$.** Can the equality-test specification be integrated with native equality transport or case reasoning so that the new arithmetic basis has a smaller complete trusted implementation? Every logical rule, arithmetic schema, parser, scope check, and declaration

validator must be charged. A validator for arbitrary claimed PR functions is not free.

2. **A syntactic presentation with fewer equality-test axioms.** Can range and reflection be replaced by a short structurally recursive definition whose whole-domain reflection theorem is provable from the tied-seed basis and equation induction? The proof must avoid assuming coverage or equality decidability before deriving them.

3. **Mechanical verification of the bounded compiler.** Formalize its cap coherence, inactive-clipping invariant, finite-case elimination of translated quantifier-free formulas, and both interpretation composites in Lean or Rocq. These are more valuable next targets than more standard-number examples.

4. **The exact source-size frontier.** Can a single well-founded explicit-parser checker improve all current byte, token, and AST coordinates under the unchanged exit-status contract? The 604/612-byte constructions do not yet do so. Compression should remain a reported observation rather than the objective.

5. **Certificate cost and readability.** The triplicate closure representation can increase serialized proof size sharply under repeated nesting. Compare complete checker complexity together with proof length, stack depth, and a readable syntax-to-certificate producer; do not count a larger untrusted compiler as a trusted-size saving.

6. **Sharp internal finite-choice complexity.** The standard-function lower bound of $k$ value pieces for $k$-ary addition is elementary and exact. Determine useful corresponding bounds on internal proof size, branch expansion for nested terms, and the cost of converting graph-defined arithmetic proofs to the native calculus.

# A Internal order, difference, and addition under a bound

This section supplies the arithmetic facts needed by the bounded compiler. All its inductions are on quantifier-free formulas with the other displayed variables held fixed as parameters. In particular, an induction below is never shorthand for induction on a universally quantified formula. Use $\epsilon$ for the numerical equality test, with values zero and one, and use the already derived terms and equations

$$P0 = 0, \quad P(Sx) = x, \qquad Z0 = 1, \quad Z(Sx) = 0, \qquad G(0, a) = 0, \quad G(Sx, a) = a,$$

$$L_h(0) = 0, \quad L_h(Sn) = h(n, L_h(n)), \qquad J(a, 0) = a, \quad J(a, 1) = Sa.$$

Here $1 = S0$. Applying $P$ to an equality of successors gives successor injectivity. Applying $Z$ to $0 = Sx$ contradicts $0 \neq 1$, so zero is distinct from every successor. The equality-test axioms consequently give

$$\epsilon(x, x) = 1, \quad \epsilon(x, y) = \epsilon(y, x), \quad \epsilon(Sx, Sy) = \epsilon(x, y), \quad \epsilon(0, Sx) = 0.$$

Each identity follows by the two possible Boolean values and the reflection law for $\epsilon$; these are derived identities, not extra axioms.

**Lemma A.1** (Derived constructor coverage)**.** *The theory proves* $x = 0 \vee x = S(Px)$.

*Proof.* Apply the already derived quantifier-free induction to $x = 0 \vee x = S(Px)$. The base is immediate. At a successor the second disjunct follows directly from $P(Sx) = x$, independently of the induction hypothesis. Thus constructor coverage need not be a primitive axiom. □

## A.1 A hit term defines a discrete linear order

Define

$$H(n,y) := L(n; r, v.J(v, \epsilon(r,y))).$$

Its defining equations are

$$H(0,y) = 0, \qquad H(Sn,y) = J(H(n,y), \epsilon(n,y)). \tag{H}$$

**Lemma A.2** (Hit identities and duality)**.** *The following identities and range assertion hold internally:*

$$\begin{aligned} &H(Sn,0) = 1, && H(Sn,Sy) = H(n,y), && H(n,n) = 0,\\ &H(Sn,n) = 1, && H(n,y) = 0 \vee H(n,y) = 1, \\ &H(Sy,x) = Z(H(x,y)). \end{aligned}$$

*Moreover a hit persists:* $H(n,y) = 1$ *implies* $H(Sn,y) = 1$*.*

*Proof.* Equation induction on $n$ proves $H(Sn,0) = 1$: its first step hits zero, and every later equality test against zero is zero. Equation induction on $n$, with $y$ fixed, proves $H(Sn,Sy) = H(n,y)$, using $\epsilon(Sn,Sy) = \epsilon(n,y)$. A further equation induction gives $H(n,n) = 0$; the recurrence then gives $H(Sn,n) = 1$.

For Boolean range use quantifier-free induction on $n$, with $y$ fixed. If $\epsilon(n,y) = 0$, the state is unchanged. If $\epsilon(n,y) = 1$, reflection gives $n = y$, so the old state is $H(n,n) = 0$ and the new state is one. If $H(n,y) = 1$, diagonal zero shows $n \neq y$, so the next equality test vanishes. This proves persistence. The same case split shows

$$H(Sn,y) = 1 \implies H(n,y) = 1 \ \vee \ n = y. \tag{New hit}$$

Finally prove duality by equation induction on $x$, with $y$ fixed. At zero both sides are one. In the successor step the new left side is $H(Sy,Sx) = H(y,x)$, whereas the old left side is $J(H(y,x), \epsilon(y,x))$. If $\epsilon(x,y) = 0$, both relevant recurrences leave their states unchanged, and the induction hypothesis gives the required equality. If $\epsilon(x,y) = 1$, put $x = y$; the new left side is zero and the new right side is $Z(H(Sx,x)) = Z1 = 0$. □

Put

$$x < y \quad :\iff \quad H(y,x) = 1, \qquad x \leq y \quad :\iff \quad H(Sy,x) = 1. \tag{Order}$$

These are quantifier-free abbreviations.

**Lemma A.3** (Discrete linear-order laws)**.** *The relations in* (Order) *form a discrete linear order with least element zero. In particular,*

$$x \leq y \iff (x = y \vee x < y), \qquad x < Sy \iff x \leq y,$$

*successor preserves and reflects both comparisons, and* $x \leq Sy$ *with* $x \neq Sy$ *implies* $x \leq y$*.*

*Proof.* Diagonal identities give reflexivity of $\leq$, irreflexivity of $<$, and $x < Sx$. The recurrence for $H(Sy,x)$, split on $\epsilon(y,x)$, gives the first displayed equivalence. Duality and Boolean range give trichotomy: if $x \neq y$, then $H(Sy,x) = H(y,x)$, and $H(x,y)$ is its Boolean complement. Thus exactly one of $x < y$, $x = y$, $y < x$ holds.

For strict transitivity retain $H(y,x)=1$ and induct on $z$ in the quantifier-free formula

$$H(z,y)=1 \;\longrightarrow\; H(z,x)=1.$$

The base has false antecedent. At the successor, the new-hit implication (New hit) gives either $H(z,y)=1$, when the induction hypothesis followed by persistence applies, or $z=y$, when the retained $H(y,x)=1$ followed by persistence applies. Transitivity of $\leq$ and the mixed comparison laws now follow by splitting equality from strict comparison. Trichotomy gives antisymmetry.

The identities $H(Sn,Sy)=H(n,y)$ give successor preservation and reflection. The equivalence $x<Sy \leftrightarrow x\leq y$ is immediate from the definitions, and the asserted discrete predecessor step follows from it and the first equivalence. Finally $H(Sx,0)=1$ gives $0\leq x$; if $x\leq 0$, trichotomy or constructor coverage gives $x=0$. □

## A.2 Difference is an internal term, not an imported operation

Define

$$d(x,y) := L(x;u,v.J(v,H(Su,y))).$$

Thus

$$d(0,y)=0, \qquad d(Sx,y)=J(d(x,y),H(Sx,y)). \tag{d}$$

**Lemma A.4** (Difference laws)**.** *The theory proves*

$$\begin{aligned} d(x,0)&=x, & d(Sx,Sy)&=d(x,y), & d(x,x)&=0,\\ d(x,y)=0 &\iff x\leq y, & d(x,Sy)&=P(d(x,y)), & d(x,y)&\leq x. \end{aligned}$$

*Difference is monotone in its first argument and antitone in its second. Furthermore*

$$\begin{aligned} y\leq x &\implies d(Sx,y)=S(d(x,y)),\\ 0<y\leq x &\implies d(x,Py)=S(d(x,y)). \end{aligned}$$

*Proof.* Equation induction gives $d(x,0)=x$ because $H(Sx,0)=1$. For simultaneous successors, induct on $x$ with $y$ fixed. The base uses $H(S0,Sy)=0$. At the successor use $H(S(Sx),Sy)=H(Sx,y)$ and the induction hypothesis in (d).

Quantifier-free induction on $x$, with $y$ fixed, proves $d(x,y)=0 \leftrightarrow H(x,y)=0$. At the successor, if $H(Sx,y)=1$, the new difference is a successor and is nonzero. If $H(Sx,y)=0$, persistence implies $H(x,y)=0$, so the old difference is zero and remains zero. Duality identifies $H(x,y)=0$ with $H(Sy,x)=1$, proving the order characterization and $d(x,x)=0$.

Now induct on $x$ for $d(x,Sy)=P(d(x,y))$, with $y$ fixed. Its successor left side is $d(x,y)$. The right side is $P(J(d(x,y),H(Sx,y)))$. When the Boolean is one, predecessor cancels the successor; when it is zero, the preceding zero characterization gives $d(x,y)=0$. Both cases give the desired equality.

Quantifier-free induction on $x$ proves $d(x,y)\leq x$: its successor state is either the old state or its successor. For first-argument monotonicity induct on $v$ in

$$u\leq v \;\longrightarrow\; d(u,z)\leq d(v,z),$$

with $u,z$ fixed. At $Sv$, either $u=Sv$ or discreteness gives $u\leq v$; in the latter case use the induction hypothesis and the fact that the new state is at least its old state. The base uses $u\leq 0\to u=0$.

For second-argument antitonicity retain $y \le z$ and induct on $x$ in $d(x,z) \le d(x,y)$. If $H(Sx,z) = 0$, the left state stays fixed and the right state cannot decrease. If $H(Sx,z) = 1$, then $z \le x$; transitivity gives $y \le x$, so both states increment and successor preserves their old comparison.

The first final implication is the defining recurrence, since $y \le x$ means $H(Sx,y) = 1$. For the second, coverage gives $y = S(Py)$ when $y > 0$. The inequalities imply $Py < x$, so $d(x,Py) \neq 0$. Functional coverage at this positive difference and $d(x,S(Py)) = P(d(x,Py))$ yield $d(x,Py) = S(d(x,y))$. □

Define

$$\mathrm{mn}(x,y) := d(x,d(x,y)).$$

**Lemma A.5** (Minimum)**.** *If $x \le y$, then $\mathrm{mn}(x,y) = x$; if $y \le x$, then $\mathrm{mn}(x,y) = y$. Consequently* mn *is the minimum operation for the internal linear order.*

*Proof.* The first claim uses $d(x,y) = 0$ and $d(x,0) = x$. For the second induct on $y$ in $y \le x \to d(x,d(x,y)) = y$, with $x$ fixed. The base is $d(x,x) = 0$. If $Sy \le x$, then $y < x$ and $0 < d(x,y) \le x$. Apply the final difference implication to obtain

$$d(x,d(x,Sy)) = d(x,P(d(x,y))) = S(d(x,d(x,y))) = Sy.$$

The last equality uses the induction hypothesis, which applies because $y \le x$. Linearity completes the interpretation of $m$. □

## A.3 Addition with an explicit common bound

Define the unary doubling term

$$\mathrm{db}(n) := L(n;u,v.S(Sv)).$$

Thus $\mathrm{db}(0) = 0$ and $\mathrm{db}(Sn) = S(S(\mathrm{db}(n)))$. The notation $2n$ in the compiler may be read as this term $\mathrm{db}(n)$; it does not invoke binary addition.

**Lemma A.6** (Doubling identities)**.** *The term* db *is monotone, $n \le \mathrm{db}(n)$, and $d(\mathrm{db}(n),n) = n$.*

*Proof.* Quantifier-free induction gives $n \le \mathrm{db}(n)$ directly from the two successors in the defining equation. Monotonicity follows by induction on the larger argument, splitting equality at its successor from the discrete predecessor case, just as for first-argument monotonicity of $d$. Equation induction proves the remaining identity. At the successor,

$$d(\mathrm{db}(Sn),Sn) = d(S(S(\mathrm{db}(n))),Sn) = d(S(\mathrm{db}(n)),n) = S(d(\mathrm{db}(n),n)) = Sn,$$

where the third equality uses $n \le \mathrm{db}(n)$. □

Put

$$K_b(a) := d(\mathrm{db}(b),d(b,a)), \qquad \mathrm{ad}_b(a,c) := d(K_b(a),d(b,c)). \qquad \text{(Bounded addition)}$$

Every expression here is a finite term of the original language.

**Lemma A.7** (Bounded-addition laws)**.** *Internally, the following hold:*

1. *If $a \le b$, then $\mathrm{ad}_b(a,0) = a$.*
2. *If $a \le b$ and $Sc \le b$, then $\mathrm{ad}_b(a,Sc) = S(\mathrm{ad}_b(a,c))$.*
3. *If $c \le b$, then $\mathrm{ad}_b(0,c) = c$.*

4. $\mathrm{ad}_b(b,b) = \mathrm{db}(b)$ *and* $\mathrm{ad}_b(a,c) \le \mathrm{db}(b)$ *for all* $a, c$.
5. $\mathrm{ad}_b$ *is monotone in its second argument.*
6. *If* $a, c \le b$ *and* $a, c \le b'$, *then* $\mathrm{ad}_b(a,c) = \mathrm{ad}_{b'}(a,c)$.

*Thus it supplies addition on inputs bounded by* $b$, *without asserting an unrestricted binary addition term.*

*Proof.* Since $d(b,a) \le b$, antitonicity and doubling give

$$b = d(\mathrm{db}(b), b) \le K_b(a) \le \mathrm{db}(b). \qquad \text{(K bounds)}$$

Also $K_b(0) = b$. If $Sa \le b$, put $q = d(b,a)$. Then $0 < q \le b \le \mathrm{db}(b)$ and $d(b,Sa) = Pq$, so

$$K_b(Sa) = d(\mathrm{db}(b), Pq) = S(K_b(a)).$$

Quantifier-free induction on $a$ now proves the first assertion under $a \le b$. The base is $d(b,b) = 0$; at the successor use the last identity and $b \le K_b(a)$ to increment $d(K_b(a), b)$.

For the second assertion put $q = d(b,c)$. The premise $Sc \le b$ gives $0 < q \le b \le K_b(a)$. Hence

$$\mathrm{ad}_b(a, Sc) = d(K_b(a), Pq) = S(d(K_b(a), q)) = S(\mathrm{ad}_b(a,c)).$$

The third assertion follows by quantifier-free induction on $c \le b$ using the first two. For the fourth, substitute $a = c = b$ and use $d(b,b) = 0$ twice; the upper bound follows from $d(K_b(a), d(b,c)) \le K_b(a) \le \mathrm{db}(b)$.

If $c \le c'$, antitonicity gives $d(b,c') \le d(b,c)$; applying antitonicity again to the outer difference gives $\mathrm{ad}_b(a,c) \le \mathrm{ad}_b(a,c')$. Finally retain $a \le b, b'$ and induct on $c$ in the quantifier-free implication

$$c \le \mathrm{mn}(b,b') \;\longrightarrow\; \mathrm{ad}_b(a,c) = \mathrm{ad}_{b'}(a,c).$$

The base gives $a$ on each side, and whenever $Sc \le \mathrm{mn}(b,b')$ the successor laws apply to both sides and preserve the induction hypothesis. □

These proofs establish the numerical prerequisites for bounded selection and clipped recursion entirely within the proposed theory. Each use of a case split is justified by numerical equality decidability, Boolean range, or the derived linear-order and coverage laws. No standard-model correctness assertion and no hidden unrestricted addition operation is used as an internal axiom.

# B Attribution of the inherited checker metatheory

The following five mathematical appendices are reproduced, with attribution, from the pinned uniform-predecessor package. They establish the scalar-iterator and Choice-DI metatheory used by the delivered small checker. Their labels and internal terminology are preserved. They do not constitute a size measurement or a proof for the new tied-seed arithmetic basis.

# C Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

## C.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only $h$ binds the state variable $u$; $n$ and $a$ do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables $x, y$ in a paired formula are distinct. An ordinary assumption $F$ is the constant clause $\langle H; v.F\rangle$, where $v$ is absent from $F$ and $H$ is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \;\; \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F} \;\; \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \;\; \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)} \;\; \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here $\varepsilon$ is a certificate choice in $\{0, 1\}$, and $\varepsilon = 1$ is permitted only when $B$ has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter $z$ is fresh for $\Gamma$, the witness $t$, and every fixed parameter of $B$. In DE the eigenparameter $a$ is fresh for $\Gamma$, the conclusion $F$, and every fixed parameter of $B$. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t, 0), [B(t, z)]_{\mathrm{at}} \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)}. \tag{Old DI}$$

The bracket supplies $B(t, z)$ exactly at a literal equation.

## C.2 Exact equivalence and the unused-witness construction

**Lemma C.1** (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

*Proof.* Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

**Theorem C.2** (Identity-language equivalence)**.** *For every retained clause context $\Gamma$ and native formula $F$,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

*The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.*

*Proof.* Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x, y)$ and its transformed premise is

$$\pi : \quad \Gamma, E(t, 0), E(t, z) \vdash E(t, Sz).$$

The outer new inference will be case DI with witness $t$. It is therefore enough to prove $E(t, Sz)$ under $\Gamma, E(t, 0)$.

Rename the eigenvariable in $\pi$ to a fresh $v$. Choose a new variable $u$ absent from $E(t, y)$ and all retained data. Under $\Gamma, E(t, 0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t, y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate $u$ is deliberately unused.

Apply DE to this paired formula. Because $u$ is unused, the opened clause is exactly

$$\langle E(t, 0); y.E(t, y) \rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t, 0)$ supplies its antecedent. Use at $Sz$ gives $E(t, Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y} E(x, y)$ with witness $t$. The original premise proof $\pi$ has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y} E(x, y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original $t$. The construction instead keeps $t$ as a fixed parameter of $E(t, y)$ and binds only the unused coordinate $u$.

**Proposition C.3** (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of $H$ followed by Use. If an old derivation has $N$ nodes and $r$ atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

*nodes.*

*Proof.* Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of $H$ needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

## C.3 Induction and numerical cases separate cleanly

**Lemma C.4** (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

*for every equation $E$, with the usual freshness condition.*

*Proof.* Choose an unused witness coordinate $u$. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at $t$. Close DE. □

**Lemma C.5** (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

*with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under* $\mathsf{D}$ *are derivable without using the progress choice.*

*Proof.* For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at $t$ finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness $a$ and introduce the new one by case DI at $a$. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

### C.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v,a,b) := I(v,a;u.b)$ with $u$ unused. Computation gives $C(0,a,b) = a$ and $C(Sv,a,b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s=t) := (C(v,0,s) = C(v,0,t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u,y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s=t,u=w) := (C(v,s,u) = C(v,t,w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A,\mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A,B),\\ M_v(E,\mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE,B),\\ M_v(\mathsf{D}_{x,y}A,E) &:= \mathsf{D}_{x,y}M_v(A,G_yE), \end{aligned}$$

where $E$ is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A,B) \dashv\vdash A$ and $M_{Sv}(A,B) \dashv\vdash B$.

Put $\delta(v) = C(v,1,0)$ and $p(z) = I(z,0;u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A,B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A,B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A,B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0=1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; $u$ is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A,B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A,B)$ and the implication gives $F$.

Given $A$ and $B$, selector endpoints and NC give $M_v(A,B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give $B$ and $A$, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\,(x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains $x$; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

## C.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let $T_{\mathrm{eq}}$ have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$
$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x\big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh $a$; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary $y$. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh $a$. Introduce the paired formula with witness $a$ by case DI: the retained base, supplied implication, and universal elimination at $Sz$ give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies $T_{\mathrm{eq}}$ with full iPRA; the present change preserves that bridge unchanged because Theorem C.2 preserves every native sequent.

### C.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

## D Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [12] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are $0$, $S$, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only $u$. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

### D.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of $C$ is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on $b$; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of $N$ is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of $Z$. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{10}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n+1) = 1, \qquad N(3n) = N(n). \tag{11}$$

For the first equation the successor step adds two to the argument of $N$; the second follows by one computation. In the third, increasing $n$ adds three to the left argument and therefore applies $Z$ once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing $M$ as remainder modulo three.

### D.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{12}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next $G$ state is $G(n) + 1$; its $M$ value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next $G$ state is $G(n) + 2$; two computations of $M$ change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n+2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{13}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{14}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (14) was proved internally; it was not inferred from an intended floor-function interpretation of $G$.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain $u$ when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{15}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (14); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

### D.3 A comparator from scalar iteration

For every fixed update $h$, equation induction on $n$, keeping $m, a$ and all parameters fixed, proves the semigroup identity

$$I(m+n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies $h$ to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update $h$, equation induction on $y$, with its initializer $a$ and all parameters of $h$ fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{16}$$

Both sides have base $h(a)$, and their successor equations apply the same $h$ to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (15), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{17}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays $a$ throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{18}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for $H$. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus $L$ has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \le y$ is motivation, not a premise of the following argument.

### D.4 Saturated iteration gives predecessor

We now need only the three equations (18). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in $B_m$ records a fixed parameter; $B_m$ is an open term abbreviation, not a function variable.

Numerical cases on $e$ prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on $e$, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (19) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (20) \end{aligned}$$

Literal-equation induction on $t$, with $m$ fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (21)$$

The base is (20). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed $m$.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (22)$$

The first uses the comparator recurrence. The second uses (21) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (23)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \qquad (24)$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on $t$, with $m$ fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (23) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

### D.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying $P$ to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

## E Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [13]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

### E.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter $n$, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments $v$ exactly when $(v + 1)^2 \le n$, and otherwise retains it. QFI in $i$, with $n$ fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals $i$, and the new state equals $Si$; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n + 1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n) + 1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed $n$ during the induction.

Now define

$$\pi(a, b) = (a + b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \leq k^2 + a < (k+1)^2$ since $a \leq k$. Root uniqueness gives $q(\pi(a,b)) = a + b$, and therefore

$$L(\pi(a,b)) = a, \qquad H(\pi(a,b)) = b.$$

Surjectivity of $\pi$ is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

### E.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}).$$

Assume by external structural induction that the earlier $g, h$ already have scalar translations $g^*, h^*$. Define

$$W(u,\vec{p}) = \pi(SL(u), h^*(L(u), H(u),\vec{p})),$$

$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u,\vec{p})), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

The update has only its state variable bound, with every $p_i$ fixed. Equation induction proves $L(V(n,\vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for $f^*$ internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

### E.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section C give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\perp$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

## F From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [11]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix D, the selector is $C(n,a,b) = I(n,a;u.b)$, addition is $x + y = I(y,x;u.Su)$, and truncated subtraction is $d(x,y) = I(y,x;u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language;

the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

## F.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions $P$ and $C$ with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{25}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{26}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x \, (x = 0 \vee x = S(Px)), \tag{27}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{28}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p} \, \left[ \big(E(0, \vec{p}) \wedge \forall n \big(E(n, \vec{p}) \longrightarrow E(Sn, \vec{p})\big)\big) \longrightarrow \forall n \, E(n, \vec{p}) \right]. \tag{29}$$

Here the schematic expression $E$ is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (29) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0, \vec{p}), \qquad \Gamma, E(z, \vec{p}) \vdash E(Sz, \vec{p}),$$

where $z$ is absent from $\Gamma$ and the passive parameters, discharge the second premise's equation and universally introduce $z$. The axiom then gives $\Gamma \vdash \forall n E(n, \vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in $\Gamma$.

## F.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{30}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{31}$$

$$K(0) = 1, \qquad K(Sx) = x. \tag{32}$$

**Constructor laws.** Successor is injective: applying $P$ to $Sx = Sy$ gives $x = y$ by (25). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (31); symmetry and (28) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

**The sign test.** For every $u$,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{33}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{34}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{35}$$

To prove the range assertion, split (27). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (26) also give, for all $u, v$,

$$C(u, v, v) = v. \tag{36}$$

Both coverage branches compute to $v$, so this identity again needs no induction.

**The countdown test and the notation $W$.** For every $u$,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{37}$$

If $u = 1 = S0$, (32) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named $W$ with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every $u$ already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are $Pu$. Thus $K$ and $W$ are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

### F.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{38}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{39}$$

Using these equations and (29), we now derive

$$0 + y = y, \tag{40}$$
$$(Sx) + y = S(x + y), \tag{41}$$
$$d(Sx, Sy) = d(x, y), \tag{42}$$
$$d(Sa, a) = 1. \tag{43}$$

Each induction below has a single equation as its induction formula.

For (40), induct on $y$. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (41), hold $x$ fixed and induct on $y$. At zero both sides are $Sx$. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (41) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{44}$$

Indeed, both sides are $S(y + r)$.

For (42), hold $x$ fixed and induct on $y$. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying $P$ to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on $a$ for (43). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (42) and the induction hypothesis.

### F.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all $x, y$:

$$C\big(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)\big) = x. \tag{45}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold $x$ fixed and induct on $y$. At $y = 0$, subtraction gives residual $x$, and (40) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (36).

For the step, assume (45) at $y$ and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for $r'$.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (45) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for $r$. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (44) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since $r'$ is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (45). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

**Reflection at residual 1.** We have, for arbitrary $a, b$,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{46}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (45). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying $P$ gives $b = a$, hence $a = b$. Conversely, $a = b$ and (43) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}\big(K(d(Sa, b))\big). \tag{47}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{48}$$

$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{49}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (34), which is equivalent to $d(Sa, b) = 1$ by (37), and then to $a = b$ by (46). The range assertion is (33) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict $a$ or $b$ to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (48) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, \big(a = b \vee (a = b \to \bot)\big) \tag{50}$$

is a theorem, not an extra hypothesis.

### F.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s, t), \qquad b_\bot = 1, \tag{51}$$

$$b_{A \wedge D} = C(b_A, b_D, 1), \tag{52}$$

$$b_{A \vee D} = C(b_A, 0, b_D), \tag{53}$$

$$b_{A \to D} = C(b_A, b_D, 0). \tag{54}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of $A$ proves, within the arithmetic theory,

$$(b_A = 0 \lor b_A = 1) \quad \text{and} \quad (b_A = 0 \longleftrightarrow A). \tag{55}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if $A$ held, the already established implication $A \to b_A = 0$ would contradict separation.

**Atomic formulas and falsity.** For an equation, both claims are (48)–(49). For falsity the range assertion is $1 = 0 \lor 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

**Conjunction.** Split $b_A = 0 \lor b_A = 1$. In the zero branch, $A$ holds and $b_{A\land D} = b_D$. Its range follows from the range of $b_D$, and its being zero is equivalent to $D$, hence to $A \land D$ under the known $A$. In the one branch, $\neg A$ holds and $b_{A\land D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \land D$; conversely an assumption $A \land D$ contradicts $\neg A$, and explosion gives $b_{A\land D} = 0$.

**Disjunction.** In the branch $b_A = 0$, the formula $A$ holds and $b_{A\lor D} = 0$. The range is immediate, and both $b_{A\lor D} = 0$ and $A \lor D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\lor D} = b_D$. Its range follows from the range of $b_D$. If its value is zero, then $D$ and hence $A \lor D$. Conversely, from $A \lor D$, the $A$ alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the $D$ alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

**Implication.** In the branch $b_A = 0$, the formula $A$ holds and $b_{A\to D} = b_D$. Its range follows from the range of $b_D$. If its value is zero, then $D$, which gives $A \to D$. Conversely, $A \to D$ and the known $A$ give $D$, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume $A$; this contradicts $\neg A$, and explosion gives $D$. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (55) for every quantifier-free formula.

### F.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context $\Gamma$ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on $z$. By (55), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term $t$. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

**Completion theorem.** In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free

induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

# G Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let $T_I$ be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from $T_I$.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section C recover the rules and axioms of $T_I$. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused $v$: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

## G.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply $P$ to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}),$$

let $\pi, L, H$ be the proved pairing and projections. The compiler uses

$$V(n, \vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

Equation induction proves $L(V(n, \vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

**Theorem G.1** (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization $T_I$ and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

*Proof.* Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

## References


[1] Vladimir Reshetnikov, *ProveIt, current iPRA drafts*, snapshot `fb2dbd2a33c6b132671a54b029d0bf3cd98c633d`, 10 November 2026. https://github.com/VladimirReshetnikov/ProveIt/tree/fb2dbd2a33c6b132671a54b029d0bf3cd98c633d/docs/drafts.

[2] *Uniform predecessor certificates for intuitionistic PRA*, `ipra_uniform_predecessor_2026-11-10.zip`, in [1]. Source of the predecessor packet, current minimum programs, and attributed interpretation appendices.

[3] *An arity calculus for iPRA*, `ipra_arity_calculus_2026-11-10.zip`, in [1]. Source of unary $D$, triplicate $Q$, and the $5 -$ arity opening argument.

[4] *Local proof scopes and interpretation obstructions for iPRA*, `ipra_local_scopes_2026-11-10.zip`, in [1], especially `theory/finite_profiles.tex`. Source of the tied-seed predecessor, gate, and zero-start macros.

[5] Michael Pinsker, *Clones containing all almost unary functions*, Algebra Universalis 51 (2004), 235–255. https://arxiv.org/abs/math/0401102.

[6] Daniel E. Severin, *Unary primitive recursive functions*, Journal of Symbolic Logic 73 (2008), 1122–1138. https://arxiv.org/abs/cs/0603063. Related unary-generation results; the bounded compiler in this report does not assume a library of unary PR primitives.

[7] *Complementary introductions with one local assumption*, inherited `reference/inherited_choice_basis.tex` from [2]; earlier source attribution is retained in that file.

[8] *Flat computation certificates for iPRA*, `ipra_flat_computation_2026-11-10.zip`, in [1].

[9] *Constructed computation assumptions for iPRA*, `ipra_constructed_computation_2026-11-10.zip`, in [1].

[10] *Generated computation assumptions for iPRA*, historical source attributed in the pinned arity-calculus package [3].

[11] *Forward-left iPRA presentation*, historical source of the inherited equality-reflection appendix, as attributed in [2].

[12] Inherited scalar predecessor development, reproduced as `reference/scalar_predecessor.tex` in [8] and [2].

[13] *Progressive-hypothesis iPRA presentation*, historical source attributed in [2].